

3) Abstraction

Definition: The process of focusing on what something does, while ignoring how it does that, just by only representing whatever is necessary.

Types of Abstraction

- **Low level Abstraction**→ Hiding implementation details. **Example**→ jab bhi hum koi 'method' call krta ha to hum uske under ke deep implementation ko nahi dekhte ha, kii us method ke ander kya ha.
- **High level Abstraction**→ Seprate what from how. By using (**Abstract class & Interfaces**). It simply means→hum ek class ki 2-child class bna dete ha. Jisko hum bolte ha **Hierarchical inheritance**
- **Abstract class** is a **keyword** jo hume parent class ke method ko sirf **declare** krne me help krta ha, taki define nahi krne pde Parent class me.
{ **accelerate();** } hum ase nahi likh skte, pr hum { **abstract accelerate();** } ase likh skte ha.
- Or jis class ka agar koi bhi **method** abstract ha to wo puri **class** ko abstract banana pdega. { **abstract class car**}. It shows the **families** of similar objects. Isme ko compulsory **contract** nahi ha.

```
// HIGH LEVEL ABSTRACTION BY "abstract" keyword.

    car c = new fuelCar();// yha pr me direct 'car' class ka object nahi bna skta, kuki car class "abstract" ha.
    c.start(); // calling to Parent class "car"
    c.accelerate();// calling to child class "fuelCar".
    c.brake(); // calling to child class "fuelCar".

}

}

abstract class car { // kuki ander "abstract methods" ha to class ko bhi "abstract" banana zaruri ha.
    void start(){
        System.out.println("car started");
    }

    abstract void accelerate(); // here "abstract" keyword is used to only declare the method for child classes.
    abstract void brake();
}

class fuelCar extends car{ // kuki Parent class me "accelerate" or "brake" methods sirf 'declare' ha
// to child class me hume 'define' bhi krna pdega.
    @Override // ye compulsory nhhi ha bs good practice ha, or ue batata ha ki hum is method ko alag se impliment kr
re ha.
    void accelerate(){
        System.out.println("accelerate fuel car");
    }

    @Override
    void brake(){
        System.out.println("fuel car is breking");
    }
}

class electricCar extends car{

    @Override
    void accelerate(){
        System.out.println("accelerate electric Car");
    }

    @Override
    void brake(){
        System.out.println("Electric car is braking");
    }
}
```

Interface class of abstraction

- **Interface** is a necessary **contract**.
- Isme **abstract** keyword lagane ki zarurat nahi hoti **declare** krne k liye.
- It gives **responsibilities, roles, capabilities**.

Therefore java ki jo apni **interfaces classes** hoti ha unke naam me “ **able**” laga hota ha→ **Example:** comparable, serializable. It shows ki ye kya kr skta ha ?

- We can only **declare** methods in parent class.
- Child class use “**implements**” keyword to join child class with parent class.
- Child class me parent class ke sabhi methods ko define krna zaruri ha kuki vo unke roles ha.
- It is a pure “**what**” language. It means ki hum class me sirf **declare** krte ha uski under detail me nahi jante ha.
- It is good practice to add “**I**” before any interface class. Taki normal class or interface class me farak pta chl sake.

```
interface Car { // interface keyword is added
    void start(); // cant define methods here.
    void accelerate();
    void brake();
}

class fuelCar implements Car { // implements keyword is added to join with Parent class.
    public void start(){
        System.out.println("Fuel Car started");// necessary to define all methods of parent
class.
    }
    public void accelerate(){
        System.out.println("Fuel Car is accelerating");
    }
    public void brake(){
        System.out.println("Fuel Car is stopping");
    }
}

class electricCar implements Car {
    public void start(){
        System.out.println("electric car is started");
    }
    public void accelerate(){
        System.out.println("electric car is accelerating");
    }
    public void brake(){
        System.out.println("electric car is stopping");
    }
}
```

Polymorphism

- Poly → many, morphism → forms.
- It means same object behaves differently on same command if one parameter is changed.
- It is possible by “method overloading”. And this is known as ‘compile-time polymorphism’.
- Jab Parent class se 3 child class ko same method mile (method @overriding) to wo run-time polymorphism hota ha.
- Fields / variables can’t be polymorphic, they don’t participate in overriding.